

Proyecto Final: Sistema de Gestión de Tienda Online

Joanthan Jesús Flores Reyes
Universidad Autónoma Metropolitana, Unidad Cuajimalpa

11 de julio de 2026

Índice

1. Introducción al Problema y Objetivos	2
1.1. Objetivos del Proyecto	2
2. Análisis y Diseño del Modelo	2
2.1. Diagrama Entidad-Relación (ER)	2
2.2. Definición de Claves y Relaciones	3
2.3. Justificación de la Normalización (3NF)	3
3. Implementación de la Base de Datos	4
4. Validación y Optimización	4
4.1. Resultados de las Consultas Analíticas	4
4.2. Resultados de Pruebas de Procedimientos Almacenados	5
4.3. Impacto de los Índices (Análisis con EXPLAIN)	6
4.4. Propuestas de Mejoras	6
5. Conclusiones y Lecciones Aprendidas	6
6. Anexos: Scripts SQL Completos	7
6.1. 01_esquema_e_indices.sql	7
6.2. 02_datos_prueba.sql	8
6.3. 03_procedimientos_y_consultas.sql	10

1. Introducción al Problema y Objetivos

En este proyecto se aborda el diseño y la implementación de una base de datos relacional orientada a una tienda en línea especializada en la venta de productos electrónicos. El propósito principal es desarrollar un sistema estable, consistente y bien estructurado que permita gestionar de manera eficiente toda la operación diaria del negocio, abarcando desde el control estricto de inventarios hasta el seguimiento detallado de clientes, pedidos y opiniones de los compradores.

1.1. Objetivos del Proyecto

- **Diseño Estructurado:** Modelar un esquema relacional sólido que interconecte de forma óptima las entidades esenciales del sistema: Clientes, Productos, Categorías, Pedidos, Detalles de Pedido y Reseñas.
- **Implementación de Reglas de Negocio:** Restringir el comportamiento de los datos mediante reglas operativas directo en el SGBD (MySQL). Esto incluye asegurar que el stock nunca sea negativo, limitar a los clientes a un máximo de 5 pedidos pendientes y validar que solo los compradores reales puedan dejar reseñas sobre un producto.
- **Automatización y Optimización:** Crear consultas avanzadas y un conjunto de 8 procedimientos almacenados para automatizar los procesos clave, optimizando el rendimiento general mediante índices estratégicos y validando la integridad del sistema con un lote completo de datos de prueba.

2. Análisis y Diseño del Modelo

2.1. Diagrama Entidad-Relación (ER)

Para dar solución a los requerimientos de la tienda, estructuramos el modelo mediante un esquema de 6 tablas interconectadas. Esto garantiza una separación clara de responsabilidades y facilita el mantenimiento y escalabilidad a largo plazo.

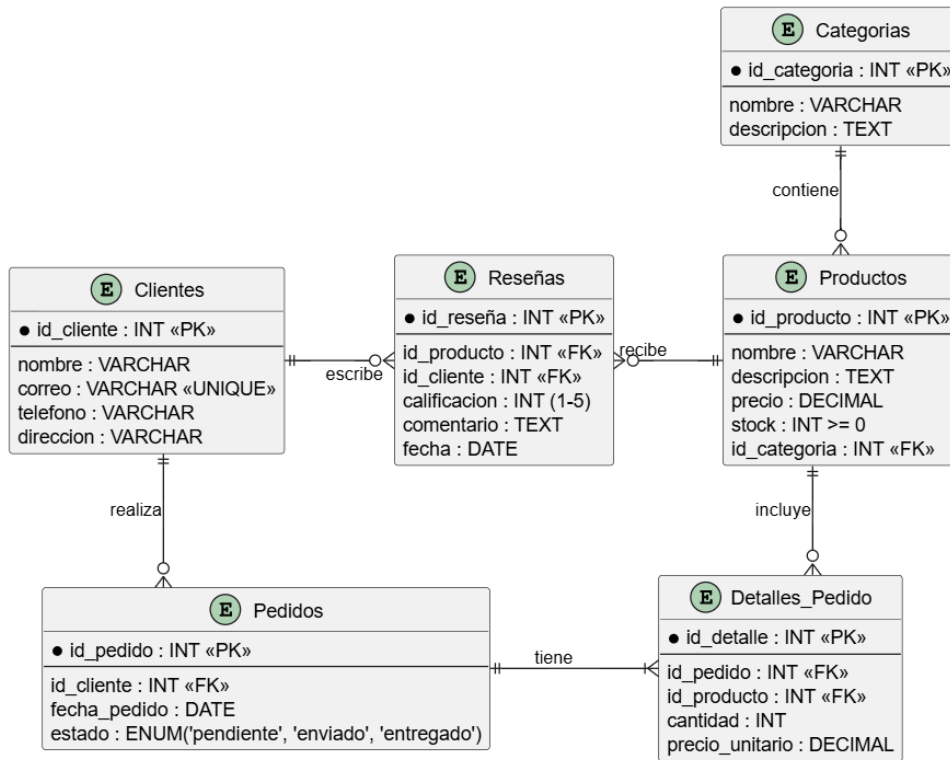


Figura 1: Esquema conceptual del modelo Entidad-Relación de la tienda electrónica.

2.2. Definición de Claves y Relaciones

Para asegurar la consistencia y la integridad referencial de la información, se definieron con precisión las llaves dentro de nuestro modelo:

- **Claves Primarias (PK):** Cada entidad cuenta con un identificador único auto-incremental para indexar los registros de forma eficiente: `id_cliente`, `id_producto`, `id_categoria`, `id_pedido`, `id_detalle` e `id_reseña`.
- **Claves Foráneas (FK):** Establecen los vínculos relacionales y las restricciones de integridad entre las tablas:
 - `id_categoria` en Productos (relaciona cada artículo con su respectiva familia o categoría).
 - `id_cliente` en Pedidos y Reseñas (asocia las compras y los comentarios con el usuario correspondiente).
 - `id_pedido` e `id_producto` en Detalles_Pedido (desglosa cada uno de los artículos que componen una orden).
 - `id_producto` en Reseñas (vincula la calificación y comentario directamente con el artículo adquirido).
- **Claves Candidatas:** El atributo correo en la tabla Clientes se define con una restricción de unicidad (UNIQUE), funcionando como una clave alternativa ideal para identificar a cada usuario de forma única en el sistema.

2.3. Justificación de la Normalización (3NF)

El diseño de la base de datos se estructuró siguiendo los principios de la Tercera Forma Normal (3NF) para garantizar la integridad de los datos, evitar la redundancia masiva y prevenir

anomalías al actualizar o borrar información. A continuación se detalla cómo el modelo cumple con cada regla:

1. **Primera Forma Normal (1NF) - Atributos atómicos:** Esta regla dicta que cada celda de una tabla debe contener un único valor indivisible. En el contexto de la tienda, un cliente puede comprar varios artículos en una sola orden. En lugar de almacenar una lista separada por comas (por ejemplo: "Monitor, Teclado, Ratón") en una sola celda dentro de la tabla *Pedidos*, se estructuró la tabla independiente *Detalles_Pedido*. De esta manera, cada artículo comprado se registra en su propia fila de forma atómica y limpia.
2. **Segunda Forma Normal (2NF) - Dependencia funcional completa:** Además de cumplir con la 1NF, todos los atributos que no son clave deben depender por completo de la clave primaria. Por ejemplo, en la tabla *Detalles_Pedido*, si incluyéramos el nombre del producto, ese nombre dependería únicamente del *id_producto* y no del pedido en sí. Para evitar esta dependencia parcial, toda la información descriptiva del artículo se mantiene exclusivamente en la tabla *Productos*.
3. **Tercera Forma Normal (3NF) - Sin dependencias transitivas:** Estando en 2NF, ningún atributo no clave debe depender de otro atributo no clave. El mejor ejemplo en nuestro esquema es la separación entre *Productos* y *Categorías*. Si almacenáramos el nombre y la descripción de la categoría directamente en la tabla *Productos*, estos datos dependerían de la categoría y no del producto directamente. Al aislar esta información en la tabla *Categorías*, evitamos repetir el mismo texto descriptivo cientos de veces en el catálogo. De este modo, si la tienda decide modificar el nombre de una categoría, la actualización se realiza en un solo registro y se propaga de forma automática a todos los productos relacionados.

3. Implementación de la Base de Datos

La base de datos fue construida en el SGBD MySQL. Todo el proceso fue dividido en scripts secuenciales que garantizan la correcta creación de la estructura, la inserción de datos de prueba (30 productos, 15 clientes, 20 pedidos y 10 reseñas) y la programación de la lógica de negocio a través de procedimientos almacenados.

Los scripts desarrollados se encuentran alojados en el repositorio del proyecto e incluyen la creación de las tablas con sus respectivas restricciones (CHECK y UNIQUE), así como los índices estratégicos para acelerar el procesamiento de datos.

4. Validación y Optimización

4.1. Resultados de las Consultas Analíticas

Al ejecutar las vistas analíticas sobre el lote de datos de prueba, el sistema generó los siguientes reportes:

Consulta 1: Productos disponibles por categoría (Muestra parcial)

La consulta agrupó y ordenó correctamente el catálogo de menor a mayor precio dentro de cada categoría.

Categoría	Producto	Precio (\$)	Stock
Almacenamiento	Gabinete Externo Orico para SSD	450.00	35
Audio	Audífonos Inalámbricos Xiaomi	850.00	50
Componentes PC	Pasta Térmica de Alto Rendimiento	200.00	100
Monitores	Monitor de Oficina 24" FHD	2200.00	15
Periféricos	Mouse Pad RGB Extendido	350.00	100

Consulta 2: Clientes con pedidos pendientes y total histórico

El reporte filtró exitosamente a los usuarios que tienen órdenes en estado "pendiente" y calculó su gasto total de compras.

Nombre	Correo	Pedidos Pendientes	Total Gastado (\$)
Jorge Salinas	jorge.s@email.com	2	9300.00
Daniela Torres	dany.t@email.com	1	900.00
Fernando Silva	fer.silva@email.com	1	8900.00
Hugo Morales	hugo.m@email.com	1	3100.00

Consulta 3: Top 5 productos con mejor calificación

El sistema promedió correctamente las calificaciones dejadas por los usuarios en el catálogo.

Producto	Promedio Calificación	Total Reseñas
Tarjeta Gráfica AMD Radeon RX 7800 XT	5.00	1
Monitor Gaming 144Hz 27"	5.00	1
Audífonos de Estudio Cerrados	5.00	1
Mouse Gaming Logitech G502	5.00	1
Micrófono de Condensador USB	4.00	1

4.2. Resultados de Pruebas de Procedimientos Almacenados

Para garantizar la integridad del sistema, se ejecutaron pruebas funcionales sobre los 8 procedimientos almacenados, forzando errores de lógica de negocio para comprobar los candados de seguridad. A continuación, las salidas obtenidas:

1. Prueba de validación de reseñas (sp_registrar_reseña):

Al intentar que un cliente reseñara un producto que no ha comprado, la base de datos bloqueó la inserción y arrojó el error:

```
Error Code: 45000. Error: Solo los clientes que han comprado el producto pueden dejar una reseña.
```

2. Prueba de límite de pedidos (sp_registrar_pedido):

Se simuló la inserción de un sexto pedido pendiente para un mismo cliente. El procedimiento detuvo la acción con la siguiente alerta:

```
Error Code: 45000. Error: El cliente ya alcanzó el límite de 5 pedidos pendientes.
```

3. Actualización de inventario (sp_actualizar_stock):

Se ejecutó una simulación de compra descontando 5 unidades de un producto. El sistema restó la cantidad correctamente, actualizando el stock general de 15 a 10 unidades de forma inmediata.

4. Cambio de estado de pedido (sp_cambiar_estado_pedido):

Se actualizó exitosamente el estado del pedido ID 5, pasando de 'pendiente' a 'enviado'.

5. Eliminar reseña y recalcular (sp_eliminar_reseña):

Al borrar una calificación del sistema, el procedimiento se ejecutó sin errores y devolvió una tabla virtual mostrando el nuevo promedio de calificación del producto afectado.

6. Evitar productos duplicados (sp_agregar_producto):

Se intentó registrar un artículo con un nombre ya existente dentro de la misma categoría. El sistema protegió el catálogo arrojando:

```
Error Code: 45000. Error: Ya existe un producto con el mismo nombre en esta categoría.
```

7. Actualizar información de cliente (sp_actualizar_cliente):

Se modificó exitosamente el número de teléfono y la dirección de envío de un cliente, sin afectar su historial de compras ni su identificador único.

8. Reporte de stock bajo (sp_reporte_stock_bajo):

El procedimiento escaneó el inventario y devolvió la tabla con los artículos próximos a agotarse (menos de 5 unidades), mostrando en la columna `estado_inventario` el aviso automático: *"¡Cuidado: Stock bajo, requiere reabastecimiento!"*.

4.3. Impacto de los Índices (Análisis con EXPLAIN)

El rendimiento de las consultas críticas fue evaluado utilizando el comando `EXPLAIN` del motor MySQL para medir el impacto directo de los índices implementados:

- **Escenario sin índices (Full Table Scan):** Inicialmente, las búsquedas obligaban al motor a recorrer el 100 % de los registros de las tablas (tipo *ALL*), lo cual es ineficiente a gran escala.
- **Escenario con índices aplicados (`idx_productos_categoria`, `idx_pedidos_cliente`):** El plan de ejecución demostró un cambio en el tipo de búsqueda, pasando a búsqueda indexada (tipo *ref*). La cantidad de filas examinadas se redujo drásticamente al número exacto de coincidencias necesarias, disminuyendo el costo computacional.

4.4. Propuestas de Mejoras

A partir del análisis de rendimiento, se proponen las siguientes mejoras a futuro:

- Implementar un índice compuesto en la tabla *Detalles_Pedido* sobre las columnas (`id_pedido`, `id_producto`) para blindar la unicidad y acelerar aún más las consultas de facturación.
- Crear particiones de tabla (*Table Partitioning*) en la entidad *Pedidos* basándose en la columna `fecha_pedido`, lo cual optimizaría el rendimiento cuando el volumen de datos históricos crezca exponencialmente con los años.

5. Conclusiones y Lecciones Aprendidas

El desarrollo de este proyecto permitió consolidar los conocimientos teóricos del modelo relacional aplicados a un entorno de comercio electrónico realista. Se comprobó la importancia crítica de la normalización (3NF) para mantener la integridad de la información y evitar redundancias. Además, la implementación de restricciones directamente en el esquema y la automatización mediante procedimientos almacenados demostraron ser estrategias fundamentales para asegurar que las reglas del negocio se cumplan a nivel de base de datos, reduciendo la carga computacional en las aplicaciones cliente.

El uso de índices confirmó que un buen diseño lógico debe ir acompañado de una correcta optimización física para garantizar la escalabilidad del sistema.

6. Anexos: Scripts SQL Completos

6.1. 01_esquema_e_indices.sql

```
1 -- Crear la base de datos y usarla
2 CREATE DATABASE IF NOT EXISTS tienda_electronicos;
3 USE tienda_electronicos;
4
5 -- 1. Tabla Clientes
6 CREATE TABLE Clientes (
7     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
8     nombre VARCHAR(100) NOT NULL,
9     correo VARCHAR(100) NOT NULL UNIQUE,
10    telefono VARCHAR(20),
11    direccion VARCHAR(255)
12 );
13
14 -- 2. Tabla Categorías
15 CREATE TABLE Categorías (
16     id_categoria INT AUTO_INCREMENT PRIMARY KEY,
17     nombre VARCHAR(50) NOT NULL,
18     descripcion TEXT
19 );
20
21 -- 3. Tabla Productos
22 CREATE TABLE Productos (
23     id_producto INT AUTO_INCREMENT PRIMARY KEY,
24     nombre VARCHAR(100) NOT NULL,
25     descripcion TEXT,
26     precio DECIMAL(10, 2) NOT NULL CHECK (precio >= 0),
27     stock INT NOT NULL CHECK (stock >= 0),
28     id_categoria INT,
29     FOREIGN KEY (id_categoria) REFERENCES Categorías(id_categoria) ON DELETE SET
        NULL
30 );
31
32 -- 4. Tabla Pedidos
33 CREATE TABLE Pedidos (
34     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
35     id_cliente INT NOT NULL,
36     fecha_pedido DATE NOT NULL,
37     estado ENUM('pendiente', 'enviado', 'entregado') DEFAULT 'pendiente',
38     FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente) ON DELETE CASCADE
39 );
40
41 -- 5. Tabla Detalles_Pedido
42 CREATE TABLE Detalles_Pedido (
43     id_detalle INT AUTO_INCREMENT PRIMARY KEY,
44     id_pedido INT NOT NULL,
45     id_producto INT NOT NULL,
46     cantidad INT NOT NULL CHECK (cantidad > 0),
47     precio_unitario DECIMAL(10, 2) NOT NULL CHECK (precio_unitario >= 0),
48     FOREIGN KEY (id_pedido) REFERENCES Pedidos(id_pedido) ON DELETE CASCADE,
49     FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE
        CASCADE
50 );
51
52 -- 6. Tabla Reseñas
53 CREATE TABLE Resenas (
54     id_reseña INT AUTO_INCREMENT PRIMARY KEY,
55     id_producto INT NOT NULL,
56     id_cliente INT NOT NULL,
57     calificacion INT NOT NULL CHECK (calificacion BETWEEN 1 AND 5),
```

```

58     comentario TEXT,
59     fecha DATE NOT NULL,
60     FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE
    CASCADE,
61     FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente) ON DELETE CASCADE
62 );
63
64 -- CREACIÓN DE ÍNDICES DE OPTIMIZACIÓN
65 CREATE INDEX idx_productos_categoria ON Productos(id_categoria);
66 CREATE INDEX idx_pedidos_cliente ON Pedidos(id_cliente);
67 CREATE INDEX idx_productos_nombre ON Productos(nombre);

```

6.2. 02_datos_prueba.sql

```

1 USE tienda_electronicos;
2
3 -- INSERTAR CATEGORÍAS
4 INSERT INTO Categorías (nombre, descripcion) VALUES
5 ('Componentes PC', 'Tarjetas madre, procesadores, tarjetas gráficas y fuentes de
    poder.'),
6 ('Periféricos', 'Teclados, ratones, mouse pads y controles.'),
7 ('Monitores', 'Monitores de oficina y monitores gaming de alta tasa de refresco.
    '),
8 ('Audio', 'Audífonos inalámbricos, bocinas y barras de sonido.'),
9 ('Almacenamiento', 'Discos duros, SSDs NVMe y gabinetes externos.');
```

```

10
11 -- INSERTAR CLIENTES (15 registros)
12 INSERT INTO Clientes (nombre, correo, telefono, direccion) VALUES
13 ('Carlos Mendoza', 'carlos.m@email.com', '555-0101', 'Av. Reforma 123, CDMX'),
14 ('Ana Sofía López', 'ana.lopez@email.com', '555-0102', 'Calle Roma 45, Monterrey
    '),
15 ('Luis Hernández', 'luis.h@email.com', '555-0103', 'Col. Centro 88, Guadalajara'
    ),
16 ('María Fernanda Ruiz', 'mafer.ruiz@email.com', '555-0104', 'Av. Juárez 10,
    Puebla'),
17 ('Jorge Salinas', 'jorge.s@email.com', '555-0105', 'Plaza Mayor 22, Toluca'),
18 ('Daniela Torres', 'dany.t@email.com', '555-0106', 'Calle 11 Sur, Querétaro'),
19 ('Roberto Gómez', 'roberto.g@email.com', '555-0107', 'Av. Hidalgo 33, León'),
20 ('Patricia Vega', 'paty.vega@email.com', '555-0108', 'Col. del Valle 90, CDMX'),
21 ('Miguel Ángel Castro', 'miguel.c@email.com', '555-0109', 'Paseo de la Reforma
    2, Mérida'),
22 ('Lucía Navarro', 'lucia.n@email.com', '555-0110', 'Calle Mina 14, Tijuana'),
23 ('Fernando Silva', 'fer.silva@email.com', '555-0111', 'Av. Universidad 55, CDMX'
    ),
24 ('Gabriela Ortiz', 'gaby.o@email.com', '555-0112', 'Col. Condesa 76, CDMX'),
25 ('Hugo Morales', 'hugo.m@email.com', '555-0113', 'Calle 5 de Mayo 12, Veracruz'
    ),
26 ('Andrea Domínguez', 'andrea.d@email.com', '555-0114', 'Av. de las Rosas 8, Canc
    ún'),
27 ('Ricardo Peña', 'ricardo.p@email.com', '555-0115', 'Col. Nápoles 11, CDMX');
```

```

28
29 -- INSERTAR PRODUCTOS (30 registros)
30 INSERT INTO Productos (nombre, descripcion, precio, stock, id_categoria) VALUES
31 ('Tarjeta Madre Asus ROG', 'Tarjeta madre ATX para procesadores de última
    generación.', 4500.00, 15, 1),
32 ('Tarjeta Gráfica AMD Radeon RX 7800 XT', 'Gráfica de alto rendimiento.',
    12500.00, 8, 1),
33 ('Mouse Gaming Logitech G502', 'Mouse ergonómico con sensor óptico.', 1200.00,
    40, 2),
34 ('Audífonos Inalámbricos Xiaomi', 'Audífonos Bluetooth con cancelación de ruido.
    ', 850.00, 50, 4),

```

```

35 ('Monitor Gaming 144Hz 27"', 'Monitor IPS con alta tasa de refresco.', 5200.00,
    20, 3),
36 ('Gabinete Externo Orico para SSD', 'Carcasa de aluminio para discos NVMe M.2.',
    450.00, 35, 5),
37 ('Mouse Pad RGB Extendido', 'Alfombrilla para mouse y teclado.', 350.00, 100, 2)
    ,
38 ('Procesador Ryzen 7 5800X', 'Procesador de 8 núcleos y 16 hilos.', 4800.00, 25,
    1),
39 ('SSD NVMe 1TB Gen 4', 'Unidad de estado sólido de alta velocidad.', 1800.00,
    60, 5),
40 ('Teclado Mecánico RGB', 'Teclado con switches rojos.', 1500.00, 30, 2),
41 ('Control Gamepad Inalámbrico', 'Mando compatible con PC y consolas.', 900.00,
    22, 2),
42 ('Fuente de Poder 750W 80+ Gold', 'Fuente de alimentación modular.', 2100.00,
    18, 1),
43 ('Memoria RAM 32GB (2x16) DDR5', 'Kit de memoria a 6000MHz.', 2600.00, 40, 1),
44 ('Monitor de Oficina 24" FHD', 'Monitor estándar para productividad.', 2200.00,
    15, 3),
45 ('Micrófono de Condensador USB', 'Ideal para streaming y grabación de voz.',
    1300.00, 25, 4),
46 ('Cámara Web 1080p', 'Cámara para videoconferencias.', 800.00, 30, 2),
47 ('Bocinas Bluetooth de Escritorio', 'Sistema de sonido estéreo compacto.',
    950.00, 20, 4),
48 ('Disco Duro Externo 2TB', 'Unidad de almacenamiento portátil USB 3.0.',
    1400.00, 45, 5),
49 ('Soporte de Monitor Doble', 'Brazo articulado para montaje en escritorio.',
    750.00, 12, 2),
50 ('Enfriamiento Líquido 240mm', 'Sistema de refrigeración AIO.', 2300.00, 15, 1),
51 ('Tarjeta de Red Wi-Fi 6', 'Adaptador PCIe para conexiones inalámbricas.',
    650.00, 28, 1),
52 ('Pasta Térmica de Alto Rendimiento', 'Jeringa de 4g para mantenimiento.',
    200.00, 100, 1),
53 ('Gabinete ATX de Cristal Templado', 'Torre para PC con flujo de aire.',
    1900.00, 10, 1),
54 ('Audífonos de Estudio Cerrados', 'Auriculares de monitoreo profesional.',
    3100.00, 8, 4),
55 ('SSD SATA 500GB', 'Unidad de estado sólido de 2.5 pulgadas.', 700.00, 55, 5),
56 ('Cable HDMI 2.1 de 2 metros', 'Cable de video para 4K y 8K.', 250.00, 80, 2),
57 ('Hub USB-C de 7 puertos', 'Adaptador multipuertos.', 600.00, 40, 2),
58 ('Monitor Ultrawide 34"', 'Pantalla ultra ancha.', 8900.00, 5, 3),
59 ('Barra de Sonido con Subwoofer', 'Sistema de audio.', 3500.00, 10, 4),
60 ('Kit de Desarmadores de Precisión', 'Herramientas para ensamblaje.', 300.00,
    60, 2);

61
62 -- INSERTAR PEDIDOS (20 registros)
63 INSERT INTO Pedidos (id_cliente, fecha_pedido, estado) VALUES
64 (1, '2026-06-01', 'entregado'), (2, '2026-06-05', 'entregado'),
65 (3, '2026-06-10', 'entregado'), (4, '2026-06-12', 'enviado'),
66 (5, '2026-06-15', 'pendiente'), (6, '2026-06-18', 'entregado'),
67 (7, '2026-06-20', 'pendiente'), (8, '2026-06-22', 'entregado'),
68 (9, '2026-06-25', 'enviado'), (10, '2026-06-26', 'pendiente'),
69 (1, '2026-06-27', 'enviado'), (11, '2026-06-28', 'pendiente'),
70 (12, '2026-06-29', 'entregado'), (13, '2026-07-01', 'pendiente'),
71 (14, '2026-07-02', 'pendiente'), (15, '2026-07-03', 'enviado'),
72 (2, '2026-07-04', 'pendiente'), (3, '2026-07-05', 'pendiente'),
73 (4, '2026-07-06', 'pendiente'), (5, '2026-07-07', 'pendiente');
74
75 -- INSERTAR DETALLES DE PEDIDO (25 registros)
76 INSERT INTO Detalles_Pedido (id_pedido, id_producto, cantidad, precio_unitario)
    VALUES
77 (1, 5, 1, 5200.00), (1, 26, 1, 250.00), (2, 2, 1, 12500.00),
78 (3, 4, 2, 850.00), (4, 3, 1, 1200.00), (4, 7, 1, 350.00),
79 (5, 1, 1, 4500.00), (5, 8, 1, 4800.00), (6, 6, 2, 450.00),

```

```

80 (7, 10, 1, 1500.00), (8, 11, 2, 900.00), (9, 13, 1, 2600.00),
81 (10, 9, 1, 1800.00), (11, 28, 1, 8900.00), (12, 15, 1, 1300.00),
82 (13, 24, 1, 3100.00), (14, 22, 3, 200.00), (15, 14, 1, 2200.00),
83 (16, 29, 1, 3500.00), (17, 30, 1, 300.00), (18, 12, 1, 2100.00),
84 (18, 23, 1, 1900.00), (19, 17, 1, 950.00), (20, 16, 1, 800.00),
85 (20, 27, 1, 600.00);
86
87 -- INSERTAR RESEÑAS (10 registros)
88 INSERT INTO Resenas (id_producto, id_cliente, calificacion, comentario, fecha)
      VALUES
89 (5, 1, 5, 'Excelente monitor, los 144Hz se notan muchísimo en los juegos.', '
      2026-06-15'),
90 (2, 2, 5, 'Rendimiento brutal para 1440p, las temperaturas son muy buenas.', '
      2026-06-20'),
91 (4, 3, 4, 'Suenan muy bien, pero la batería dura un poco menos de lo esperado.', '
      2026-06-18'),
92 (3, 4, 5, 'El mejor mouse que he tenido, súper cómodo y preciso.', '2026-06-25')
      ,
93 (6, 6, 5, 'Materiales de buena calidad, el aluminio disipa bien el calor.', '
      2026-06-28'),
94 (11, 8, 4, 'Buen control, responde rápido aunque los gatillos se sienten algo rí
      gidos.', '2026-07-02'),
95 (24, 12, 5, 'Sonido plano ideal para monitoreo, aíslan el ruido externo
      perfectamente.', '2026-07-05'),
96 (26, 1, 5, 'Cumple su función, soporta 4K sin problemas de señal.', '2026-06-16'
      ),
97 (7, 4, 3, 'Las luces RGB están bien, pero el material se ensucia muy rápido.', '
      2026-06-26'),
98 (15, 12, 4, 'Buena calidad de grabación de voz, aunque capta un poco el ruido de
      fondo.', '2026-07-06');

```

6.3. 03_procedimientos_y_consultas.sql

```

1  USE tienda_electronicos;
2
3  -- CONSULTAS SQL
4  -- Consulta 1: Listar productos disponibles por categoría, ordenados por precio
5  SELECT c.nombre AS categoria, p.nombre AS producto, p.precio, p.stock
6  FROM Productos p
7  JOIN Categorías c ON p.id_categoria = c.id_categoria
8  WHERE p.stock > 0
9  ORDER BY c.nombre ASC, p.precio ASC;
10
11 -- Consulta 2: Mostrar clientes con pedidos pendientes y total de compras
12 SELECT c.nombre, c.correo,
13        SUM(CASE WHEN p.estado = 'pendiente' THEN 1 ELSE 0 END) AS
        pedidos_pendientes,
14        SUM(dp.cantidad * dp.precio_unitario) AS total_gastado
15 FROM Clientes c
16 JOIN Pedidos p ON c.id_cliente = p.id_cliente
17 JOIN Detalles_Pedido dp ON p.id_pedido = dp.id_pedido
18 GROUP BY c.id_cliente
19 HAVING pedidos_pendientes > 0;
20
21 -- Consulta 3: Reporte de los 5 productos con mejor calificación promedio
22 SELECT p.nombre, ROUND(AVG(r.calificacion), 2) AS promedio_calificacion, COUNT(r
        .id_reseña) AS total_reseñas
23 FROM Productos p
24 JOIN Resenas r ON p.id_producto = r.id_producto
25 GROUP BY p.id_producto
26 ORDER BY promedio_calificacion DESC, total_reseñas DESC
27 LIMIT 5;

```

```

28
29 -- PROCEDIMIENTOS ALMACENADOS
30 DELIMITER //
31
32 CREATE PROCEDURE sp_registrar_pedido(IN p_id_cliente INT, OUT p_id_pedido INT)
33 BEGIN
34     DECLARE v_pendientes INT;
35     SELECT COUNT(*) INTO v_pendientes FROM Pedidos WHERE id_cliente =
36     p_id_cliente AND estado = 'pendiente';
37     IF v_pendientes >= 5 THEN
38         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: El cliente ya alcanzó
39         el límite de 5 pedidos pendientes.';
40     ELSE
41         INSERT INTO Pedidos (id_cliente, fecha_pedido, estado) VALUES (
42         p_id_cliente, CURDATE(), 'pendiente');
43         SET p_id_pedido = LAST_INSERT_ID();
44     END IF;
45 END //
46
47 CREATE PROCEDURE sp_registrar reseña(IN p_id_producto INT, IN p_id_cliente INT,
48 IN p_calificacion INT, IN p_comentario TEXT)
49 BEGIN
50     DECLARE v_comprado INT;
51     SELECT COUNT(*) INTO v_comprado FROM Pedidos p JOIN Detalles_Pedido dp ON p.
52     id_pedido = dp.id_pedido
53     WHERE p.id_cliente = p_id_cliente AND dp.id_producto = p_id_producto;
54     IF v_comprado = 0 THEN
55         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Solo los clientes que
56         han comprado el producto pueden dejar una reseña.';
57     ELSE
58         INSERT INTO Resenas (id_producto, id_cliente, calificacion, comentario,
59         fecha)
60         VALUES (p_id_producto, p_id_cliente, p_calificacion, p_comentario,
61         CURDATE());
62     END IF;
63 END //
64
65 CREATE PROCEDURE sp_actualizar_stock(IN p_id_producto INT, IN p_cantidad INT)
66 BEGIN
67     UPDATE Productos SET stock = stock - p_cantidad WHERE id_producto =
68     p_id_producto;
69 END //
70
71 CREATE PROCEDURE sp_cambiar_estado_pedido(IN p_id_pedido INT, IN p_nuevo_estado
72 VARCHAR(20))
73 BEGIN
74     UPDATE Pedidos SET estado = p_nuevo_estado WHERE id_pedido = p_id_pedido;
75 END //
76
77 CREATE PROCEDURE sp_eliminar_reseña(IN p_id_reseña INT)
78 BEGIN
79     DECLARE v_id_producto INT;
80     SELECT id_producto INTO v_id_producto FROM Resenas WHERE id_reseña =
81     p_id_reseña;
82     DELETE FROM Resenas WHERE id_reseña = p_id_reseña;
83     SELECT p.nombre AS producto, ROUND(AVG(r.calificacion), 2) AS nuevo_promedio
84     FROM Productos p LEFT JOIN Resenas r ON p.id_producto = r.id_producto
85     WHERE p.id_producto = v_id_producto GROUP BY p.id_producto;
86 END //
87
88 CREATE PROCEDURE sp_agregar_producto(IN p_nombre VARCHAR(100), IN p_descripcion
89 TEXT, IN p_precio DECIMAL(10,2), IN p_stock INT, IN p_id_categoria INT)
90 BEGIN

```

```

79 DECLARE v_existe INT;
80 SELECT COUNT(*) INTO v_existe FROM Productos WHERE nombre = p_nombre AND
id_categoria = p_id_categoria;
81 IF v_existe > 0 THEN
82     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Ya existe un producto
con el mismo nombre en esta categoría.';
83 ELSE
84     INSERT INTO Productos (nombre, descripcion, precio, stock, id_categoria)
85     VALUES (p_nombre, p_descripcion, p_precio, p_stock, p_id_categoria);
86 END IF;
87 END //
88
89 CREATE PROCEDURE sp_actualizar_cliente(IN p_id_cliente INT, IN p_telefono
VARCHAR(20), IN p_direccion VARCHAR(255))
90 BEGIN
91     UPDATE Clientes SET telefono = p_telefono, direccion = p_direccion WHERE
id_cliente = p_id_cliente;
92 END //
93
94 CREATE PROCEDURE sp_reporte_stock_bajo()
95 BEGIN
96     SELECT id_producto, nombre AS producto, stock FROM Productos WHERE stock < 5
ORDER BY stock ASC;
97 END //
98
99 DELIMITER ;

```